OPTIONS PRICING AND PAPER-TRADING ENGINE

AURA-OPT Architecture

One Java program that reads option prices, decides whether to trade, sends orders to a practice account, and keeps books on the result. A web page shows all of it.

The short version

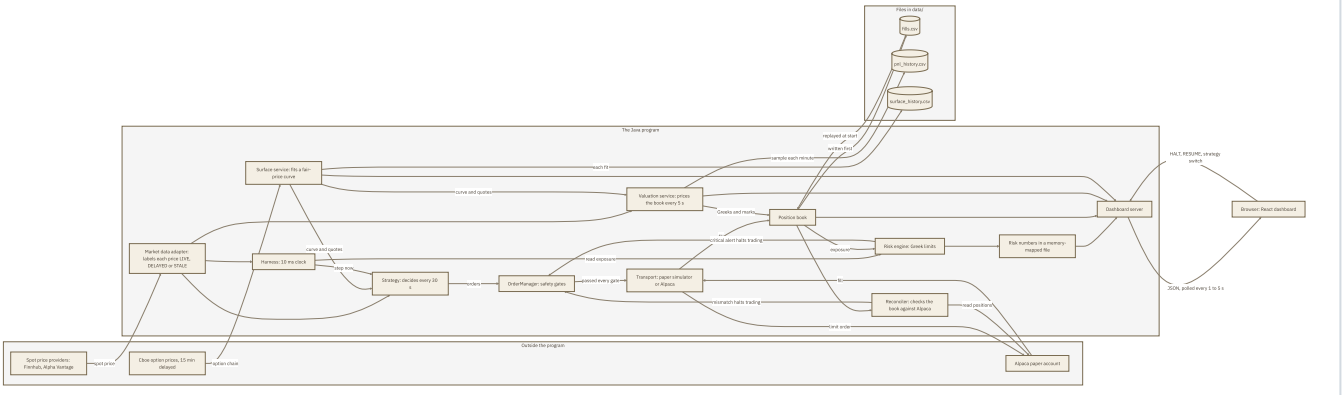
ONE PROCESS
Everything runs in a single JVM. There is no database, no message queue and no second service.

FIVE CLOCKS
Risk tick every 10 ms, valuation every 5 s, strategy every 30 s, quote reload every 60 s, curve refit every 15 min.

THREE FILES
State lives in CSV files under `data/`: fills, P&L samples and surface fits. Restart rebuilds the book from the fills.

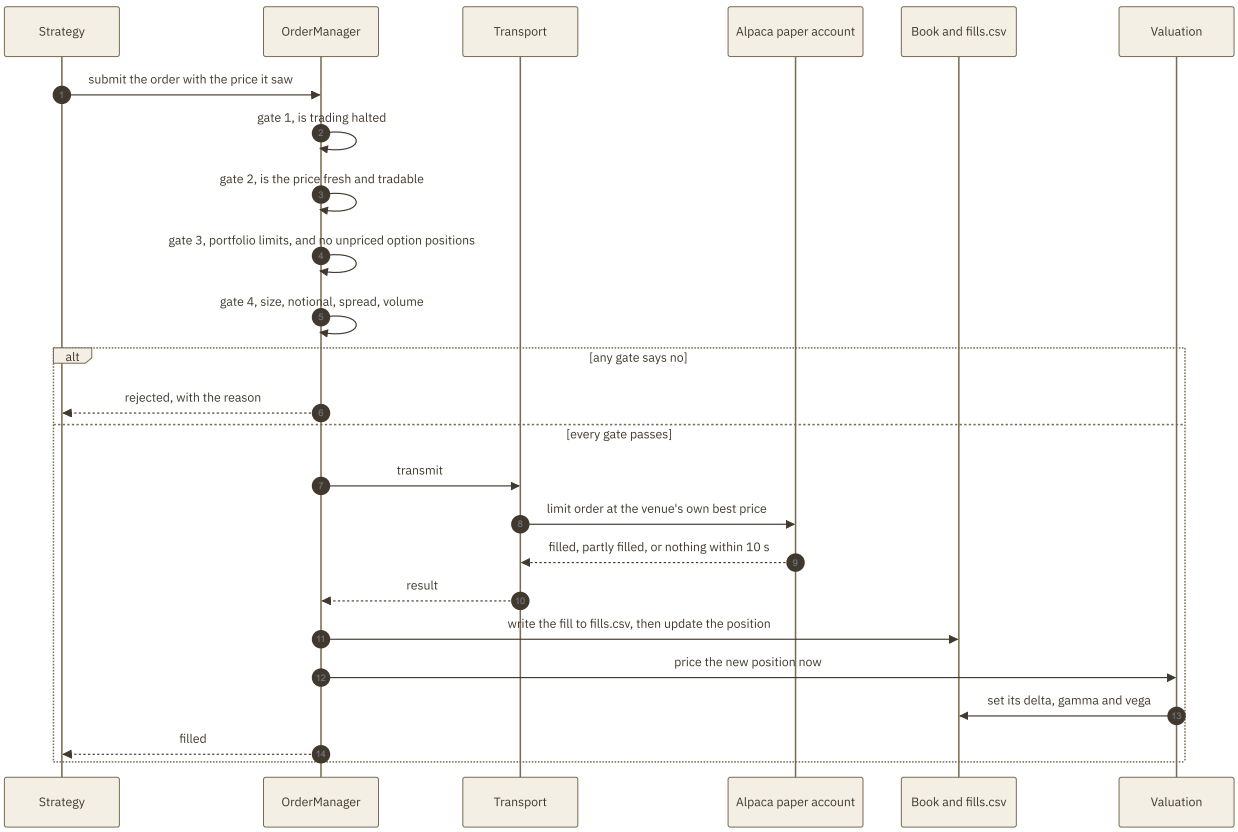
FAKE MONEY
Orders go to an Alpaca paper account. The client can only reach Alpaca's paper address.

1. What talks to what



Read it left to right. Prices come in on the left, decisions flow through the safety gates in the middle, and everything ends up in the book and on the dashboard. Two arrows lead back into the safety gates: the risk engine and the reconciler can each stop all trading.

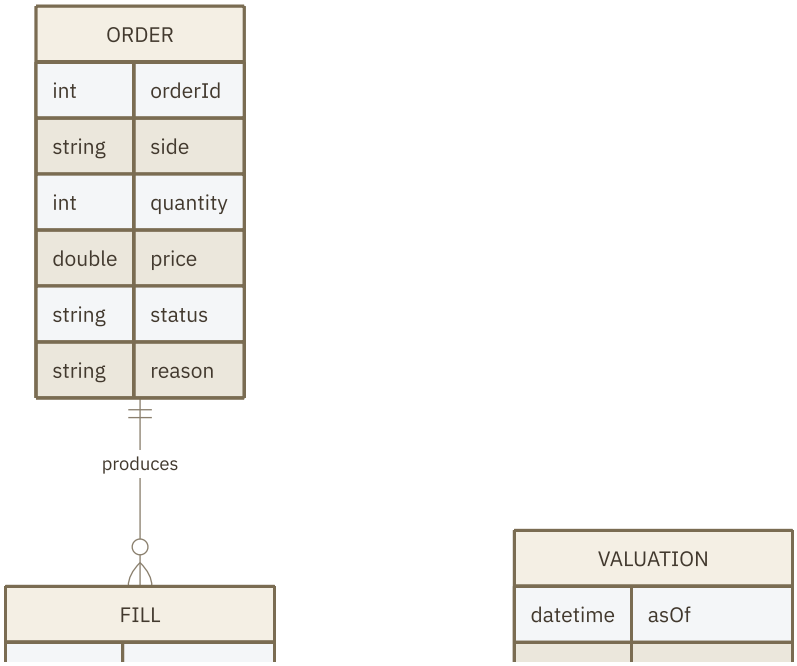
2. The life of one order

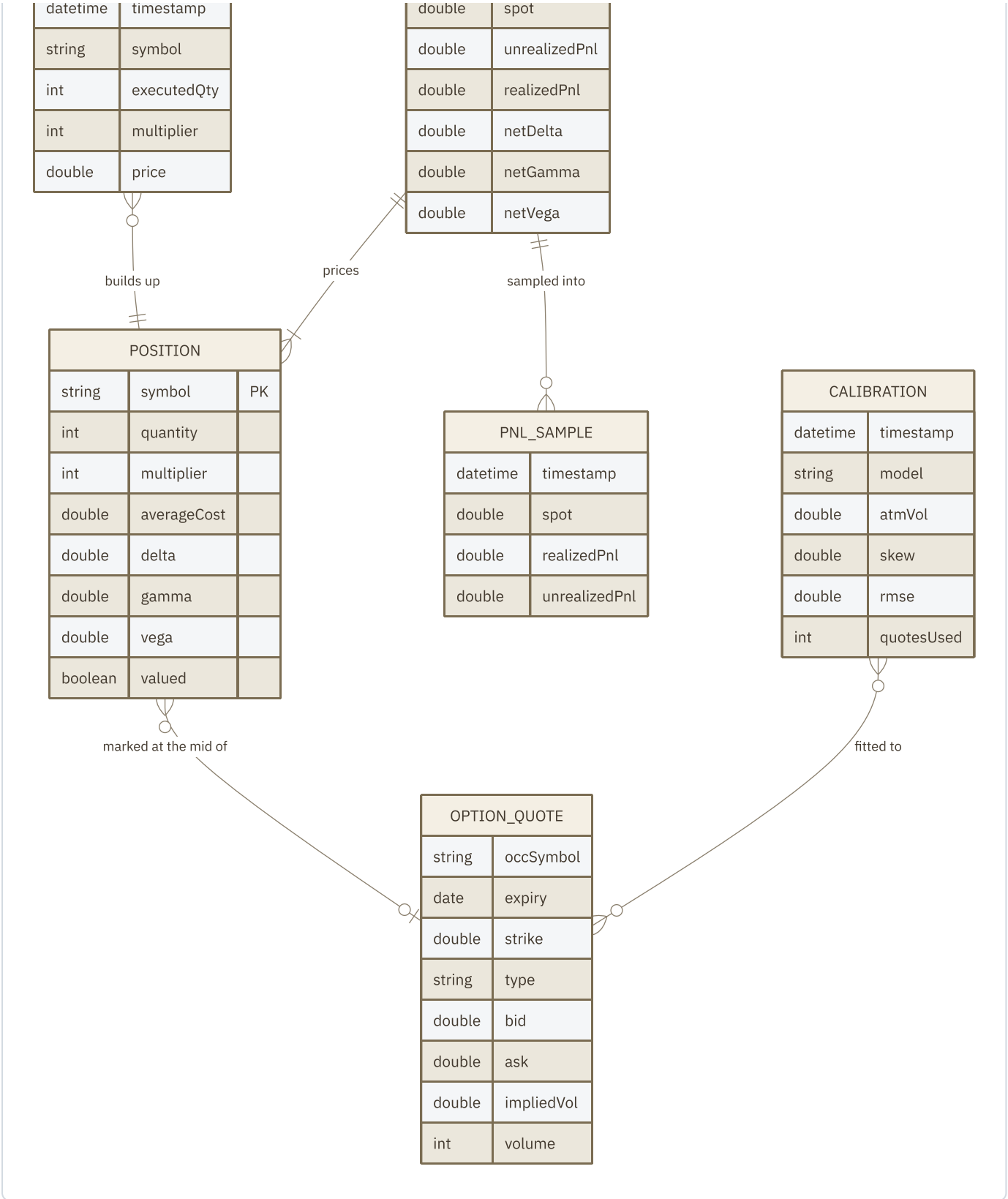


The order of the gates matters: the cheap yes-or-no checks come first and the venue is only contacted once everything passes. The fill is saved to disk before the in-memory book changes, so a crash can never leave memory ahead of the record.

3. The data

There is no database, so this is a data model rather than a true ERD. Three entities are CSV files on disk. The others live in memory and are rebuilt or refetched on start.





On disk: FILL is `fills.csv`, PNL_SAMPLE is `pnl_history.csv`, CALIBRATION is `surface_history.csv`. In memory: ORDER (last 10,000 kept), POSITION (rebuilt by replaying fills), OPTION_QUOTE (reloaded from Cboe every minute) and VALUATION (recomputed every 5 s).

A CSV row has no key and no foreign key. The links in the diagram are conceptual: a position is simply the sum of the fills with its symbol, and the order-to-fill link exists in the logs and the order tape, not in the file.

4. What happens at start-up

1. Read `config.yaml` and `.env`, and stop with a clear message if anything is wrong.
2. Replay `fills.csv` to rebuild every position, average cost and realised profit.
3. If the transport is Alpaca, compare that book with the account. A mismatch halts trading before any order can be sent.
4. Start the clocks: risk tick, surface fits, valuation, reconciler, then the dashboard on port 8080.
5. The strategy waits until it has a fresh quote and a fitted curve, and says why in its decision line while it waits.

5. Where each box lives in the code

Paths are under `src/main/java/com/sbk/optionspricer/` unless they start with `web-react`. A good reading order is the first row, then rows 5, 6 and 8.

BOX	WHAT IT DOES	FILE
Wiring	Builds every component and connects them. Read this first to see the whole system.	`AppCompositionRoot.java, Main.java`
Option prices	Downloads Cboe's delayed chain and turns it into quotes.	`CboeOptionChain.java`
Surface service	Every 15 min fits the curve, every minute reloads quotes, labels where the data came from.	`core/VolatilitySurfaceService.java, volatility/SurfaceFitter.java`
Spot prices	Asks the providers in order, with budgets and back-off, and never invents a missing field.	`web/LiveSpotProvider.java, market/LiveMarketSnapshotAdapter.java`
Strategy	Compares the straddle's market volatility with the curve and decides buy, sell, hold or close.	`execution/VolSpreadStrategy.java`
Clock	Runs the 10 ms tick and hands strategy steps to their own thread.	`core/QuantSimulationHarness.java`
Safety gates	Halt, data freshness, portfolio limits, size and liquidity	`execution/OrderManager.java, PortfolioRiskAdmission.java,`

	checks, then the transport.	`PreTradeRiskFilter.java, TradingHalt.java`
Book and fills	Holds positions and average cost, and writes each fill to disk first.	`execution/PositionTracker.java, risk/FillLedger.java`
Venues	The in-process simulator, or limit orders to the Alpaca paper account.	`execution/PaperTradingExecutionAdapter.java, AlpacaPaperTransport.java, AlpacaPaperClient.java`
Valuation and P&L	Prices the book every 5 s, computes Greeks, and records profit and drawdown.	`core/PortfolioValuationService.java, execution/PnlHistory.java`
Risk engine	Totals the book's exposure, raises alerts at the limits, publishes the numbers.	`core/UnifiedQuantEngine.java, risk/GreekAlertManager.java`
Reconciler	Every minute compares local positions with Alpaca's and halts on a difference.	`execution/BookReconciler.java`
Dashboard	HTTP API with login, plus the React page that polls it.	`web/OptionsDashboardServer.java, web-react/src/App.jsx`
Pricing maths	Black-Scholes prices and Greeks, and the implied-volatility solver.	`BlackScholesPricer.java, ImpliedVolatilitySolver.java`

Words you will meet

Option
A contract whose price depends on where SPY ends up and how much it swings.

Straddle
A call and a put at the same strike. It pays when the price moves a lot in either direction, so it is a bet on size of move, not direction.

Greeks
How much a position's value changes when the price, the swinginess or time moves. Delta is the main one.

Ledger
The file `fills.csv`, one row per fill. It is the record

Implied volatility
The amount of swinging an option's price implies. The program works backwards from price to this number.

Surface, or curve
A smooth fit through all the implied volatilities. The strategy treats it as a fair value and trades the gap.

Paper account
A practice account at a broker. Orders fill like real ones but the money is fake.

Halt
A switch that rejects every new order. Risk alerts,

the book is rebuilt from.

a book mismatch or a person can trip it, and only a person clears it.